D2 · Process: clean records with Spark

Normalize, filter and count words twice: once in plain Python, once as a real Spark job that matches it.

COMP 329 course staff

Rice University

Spring 2027 · Assignment D2

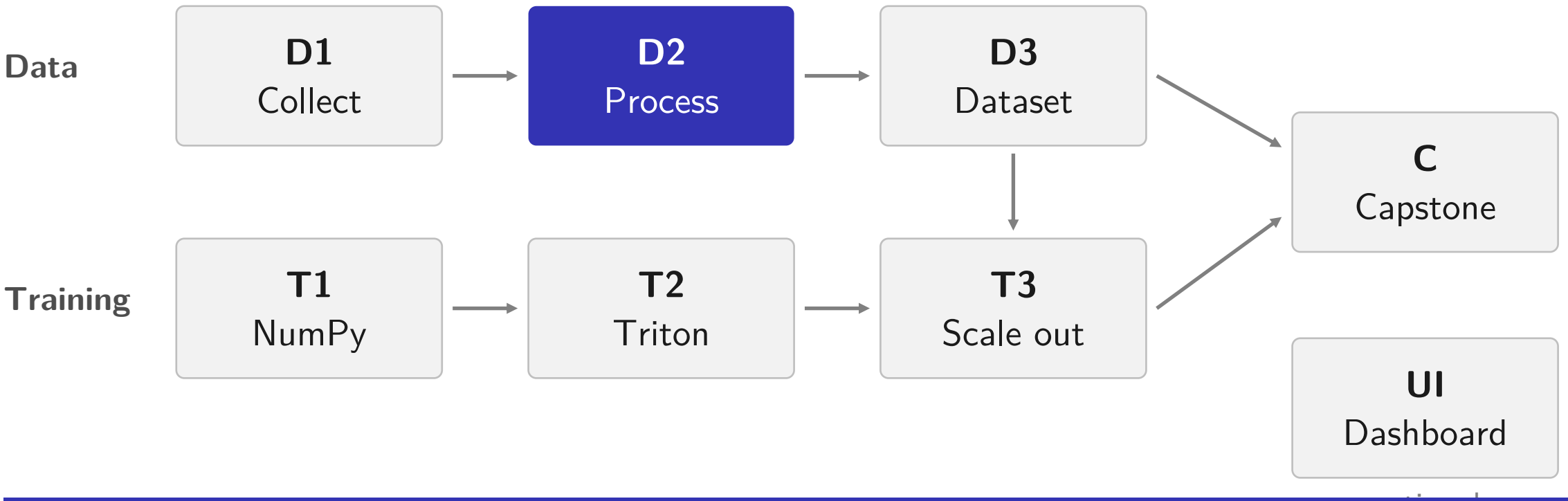
Release week 3

Checkpoint week 4

Final week 5

8-10 hours

Where this assignment fits



In our pipeline

This assignment: `articles.parquet` → **D2** → `processed/*.parquet`

What you will build, and why

Decide which articles are good enough to train on, explain every decision, and do it at Spark scale.

1 **Two paths, one answer**

A readable local reference proves what the Spark job must produce.

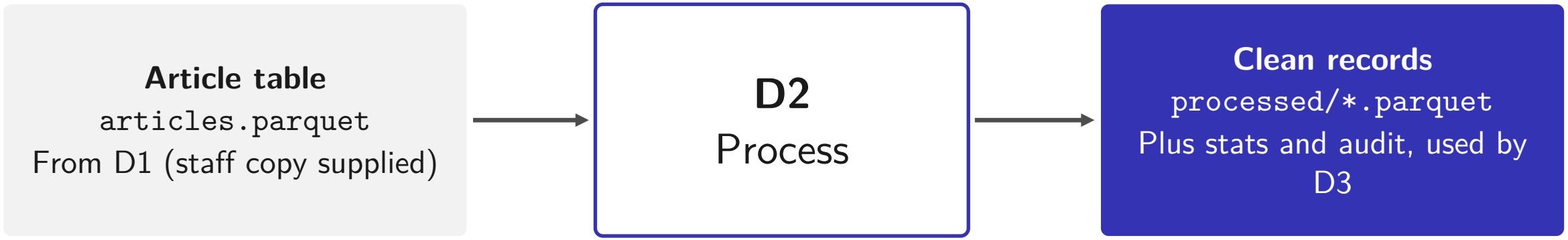
2 **Real distribution**

Partitions, lazy plans and a shuffle, not a Python loop on the driver.

3 **Auditability**

One audit row per input says why it was kept or removed.

What goes in, what comes out



Files you produce

`processed/*.parquet`
Kept, normalized records

`processing.stats.json`
Counts, bytes, top 12 words, plan details

`processing-audit.jsonl`
One row per input, with reason and previews

Normalize first, then check quality

```
rules, first match wins:
  short        < 120 characters
  long         > max (0 = off)
  email        contains an address
  low_english_letter_ratio < 0.55
  keep

len('café')                  → 4
len('café'.encode('utf-8')) → 5
```

1. **Long English paragraph**

   → keep

2. **"Tiny."**

   → short

3. **Paragraph with analyst@example.org**

   → email, or [EMAIL] under redaction, then the other rules

4. **Characters ≠ bytes**

   Record UTF-8 bytes and mean characters separately.

How Spark actually runs your job

1 **Read + repartition**
Parquet input split into the configured partitions.

→

2 **Transform (lazy)**
Columns and UDFs only build a plan.

→

3 **Explode words**
One row per word occurrence.

→

4 **Group = shuffle**
Each word must reach one aggregation worker.

→

5 **Action**
count, collect or write finally runs the plan.

Key idea

Stop timers only after an action. Cache only frames reused by several actions, then unpersist them.

The staff input after processing

34	32	2
input records	kept	removed

Key idea

One short article and one email-containing article are removed. Local and Spark results must agree on records, bytes, character means and top words.

What you write, what you get

You write

`process(source, output, config, spark=None)`

`starter/process.py` · 5 numbered TODOs

Provided

- `prepare_text` (normalization, redaction)
- `quality_reason` (the filter rules)
- SCHEMA, `spark_session`, `spark_plan`
- JSON helpers and public tests

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1 — 2 — 3 — 4 — 5

Local rules

Check IDs, normalize, classify, audit.

Local stats

Totals and top words with Counter.

Spark job

Same rules as DataFrames and UDFs.

Parity

Spark output matches the local path.

Studies

Strong, weak and partition scaling.

```
check d2 --checkpoint
```

```
check d2 --systems
```

```
lab.py d2
```

Check

Done when: All 4 tests pass with --systems, and 34 → 32 is explained.

If something goes wrong

Spark says Java is missing
Check: Run preflight; check `JAVA_HOME`.

café counts differ
Check: Both paths must use the same Unicode word rule.

Email appears in a preview
Check: Mask previews, even for rejected rows.

Timing is implausibly tiny
Check: Building a plan is not running it; add an action.

Repeated jobs eat memory
Check: Unpersist cached frames and stop the session.

Driver loop over rows
Check: Doesn't count; transform and aggregate in Spark.

The performance study

When do more Spark workers actually help?

Predict first

Predict startup, shuffle and small-partition overhead before measuring.

Workers 1 / 2 / 4
strong scaling, 48 articles

24 / 48 / 96 articles
weak scaling

Partitions 1 / 2 / 4
two workers

How to measure: One warmup + three repeats, fresh outputs. A slower result is still useful evidence; keep it.

How you are graded

20	40	16	24
Checkpoint	Final correctness	Performance	Analysis

Correctness: rules/identity 10 · statistics 10 · actual Spark/parity 12 · audit/artifacts 8

Check

Done when: All 4 tests pass with --systems, and 34 → 32 is explained.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py inputs --output runs/inputs
$ python course.py check d2 --checkpoint
$ python lab.py d2 --input runs/inputs/articles.parquet \
    --output runs/d2-spark --workers 2
```

Your folder

`02_d2_spark/`

Handout:
`02_d2_spark/README.md`